

IIIT BHOPAL METAVERSE

PROJECT REPORT

Project Design (CSE 5005) – 5th Semester

Submitted in partial fulfillment of the academic requirements
for the Bachelor of Technology degree
Department of Computer Science and Engineering

Submitted by
Priyanshu Jain (24U021020)
Akshat Lomas (24U021023)

Under the supervision of
Dr. Supriyo Mandal
Assistant Professor
Department of Computer Science and Engineering

Indian Institute of Information Technology Bhopal
November 2026

CERTIFICATE

Thisis to certify that the project report entitled “IIIT BHOPAL METAVERSE” has been prepared by Priyanshu Jain (Scholar No. 24U021020) and Akshat Lomas (Scholar No. 24U021023), students of the Department of Computer Science and Engineering in Indian Institute of Information Technology, Bhopal, as part of Project Design (CSE 5005), 5th Semester. The work has been carried out under the supervision of Dr. Supriyo Mandal, Assistant Professor, Department of Computer Science and Engineering.

To the best of our knowledge, this report presents the genuine project work, architectural implementation, route-finding algorithms, and interactive virtual environment developed during this academic semester. Formal institutional approval and signatures are to be completed by the concerned authorities.

Date: November 2026
Bhopal (M.P.)

Supervisor:

Dr. Supriyo Mandal
Assistant Professor
Department of CSE, IIIT Bhopal

Department Coordinator:

Dr. Rahul Chaurasiya
Head / Department Coordinator
Department of CSE, IIIT Bhopal

DECLARATION

We, Priyanshu Jain (Scholar No. 24U021020) and Akshat Lomas (Scholar No. 24U021023), hereby declare that the project report entitled “IIIT BHOPAL METAVERSE” is submitted as part of Project Design (CSE 5005), 5th Semester, in the Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal.

We declare that this report has been prepared from our original design and development work, source code implementations, and the technical literature cited throughout the document. External documentation, libraries, and published works have been properly acknowledged through formal citations. Any content requiring institutional verification, including implementation demonstrations, testing logs, and originality verification, has been prepared honestly in accordance with institute requirements.

We further declare that the project report has not been submitted in whole or in part for the award of any other degree or diploma in this or any other educational institution.

Priyanshu Jain (24U021020) **Akshat Lomas (24U021023)**

Signature: _____ Signature: _____

ACKNOWLEDGEMENT

Weexpress oursincere gratitude to Dr.Supriyo Mandal, Assistant Professor, Department of Computer Science and Engineering, IIIT Bhopal, for his continuous supervision, insightful technical feedback, and academic guidance throughout the design and development of the IIIT BHOPAL METAVERSE project. His suggestions regarding client-server modularity, pathfinding efficiency, and robust failure recovery were instrumental in refining our system architecture.

We thank Dr. Rahul Chaurasiya, Head and Department Coordinator, and the Department of Computer Science and Engineering, Indian Institute of Information Technology, Bhopal, for providing the supportive academic and laboratory environment from which this project was undertaken. We also extend our gratitude to the open-source software communities and standards organizations whose foundational tools and specifications supported our implementation.

Finally, we thank our families, batchmates, and friends for their encouragement and patience during our project work. Any remaining errors, omissions, or shortcomings in this report remain our own responsibility.

Priyanshu Jain (24U021020)
AkshatLomas (24U021023)
Department of Computer Science and Engineering
IIIT Bhopal

ABSTRACT

IIITBHOPALMETAVERSE is a browser-accessible interactive software application created to help students, faculty members, and campus visitors explore selected areas of the IIIT Bhopal campus. Rather than relying on disconnected static floor plans or generic satellite maps, users can control a custom student avatar within a responsive 2.5D virtual environment. The application brings spatial navigation, comprehensive facility information, and institutional directory lookups into a unified, lightweight interface.

On the client side, the frontend integrates React 18 with the Phaser 3 game engine through an asynchronous decoupled event bus. React manages high-level routing, search overlays, heads-up displays, and detailed modal views, while Phaser handles world rendering, player movement, physics collisions, and camera tracking. Campus buildings are rasterized onto an internal coordinate grid where building boundaries act as solid colliders, preventing the avatar from passing through walls while allowing free exploration across pathways.

The backend is built with Node.js and Express, exposing structured REST endpoints for campus configuration, location records, spatial search, and route calculation. It includes a resilient dual-mode architecture that operates in a zero-dependency in-memory demo mode when a database connection is absent, while seamlessly integrating with MongoDB via Mongoose when configured. The system incorporates an A* pathfinding engine that computes collision-free walking trajectories, calculates realistic physical distances in meters, and estimates transit times at human walking speeds.

In addition to spatial exploration, the system includes an intelligent conversational assistant capable of executing map actions, along with real-time virtual classroom modules supporting WebSocket events and LiveKit WebRTC media streams. All 16 automated integration and unit test cases pass consistently in local verification runs. Keywords: virtual campus, metaverse, 2.5D environment, A* pathfinding, React, Phaser, Express, MongoDB, AI assistant, virtual classroom.

TABLE OF CONTENTS

Title Section		Page No.
Certificate		2
Declaration		3
Acknowledgement		4
Abstract		5
Introduction		8
Literature Review		10
Problem Definition and Objectives		13
Proposed Methodology and Work Description Proposed Algorithms		16
Flowcharts, Block Diagrams and DFD		21
Tools, Technologies and Implementation		28
Result Discussion and Analysis		31
Conclusion and Future Scope		34
References		35
List of Publications		37
Appendix A – Submission Checklist		37
Appendix B – Repository and Reproducibility Notes		37

LIST OF FIGURES

Figure No.	Description	Page No.
Figure 4.1	System architecture of IIIT BHOPAL METAVERSE	8
Figure 6.1	Use case overview	15
Figure 6.2	DFD Level 0 – context diagram	16
Figure 6.3	DFD Level 1 – core request processing	16
Figure 6.4	Sequence diagram – campus navigation request	17
Figure 6.5	Conceptual MongoDB collection model	17
Figure 6.6	Activity flow – student uses virtual campus	18
Figure 6.7	A* route-finding flowchart	19
Figure 6.8	AI campus assistant request flow	20
Figure 6.9	Virtual classroom session flow	21

LIST OF TABLES

Table No.	Description	Page No.
Table 2.1	Comparison of approaches to campus exploration	5
Table 3.1	Functional requirements	6
Table 3.2	Non-functional requirements	7
Table 4.1	Campus coordinate and grid model	9
Table 4.2	Project modules and responsibilities	11
Table 5.1	A* symbols and definitions	12
Table 7.1	Technology stack based on package manifests and source files	22
Table 7.2	Backend API groups	23
Table 8.1	Functional evidence inventory	25
Table 8.2	Comparative analysis framework	26
Table 8.3	Known limitations and implications	27

1. INTRODUCTION

1.1 Background

A modern university campus incorporates dozens of physically separated facilities, including academic lecture halls, departmental computing laboratories, libraries, administrative cells, and student hostels [22]. When new students or external visitors arrive at an institute, understanding the layout of these buildings and learning how to navigate between them is often challenging [22]. Conventional static layout maps provide an overview of the grounds, but they fail to support search, interactive route planning, or real-time spatial exploration in an accessible format [25].

The term metaverse is widely applied across academic research and computing literature to denote persistent or interactive digital environments where users engage with spatially organized information and digital avatars [1], [2]. In this project, the concept is applied to create an accessible, browser-rendered 2.5D representation of selected facilities at IIIT Bhopal [8]. Rather than demanding heavy proprietary virtual reality headsets or computationally demanding 3D gaming hardware, the platform focuses on delivering practical, responsive campus exploration on standard consumer computers [8], [23].

1.2 Project Overview

The application architecture is organized into two primary workspaces: client and server [16]. The frontend client is developed with React 18, Vite, React Router, Tailwind CSS, and the Phaser 3 gaming framework [3], [4], [14], [15]. React governs top-level page views, navigation bars, search inputs, and modal interfaces, while Phaser manages the 2.5D canvas, sprite movements, collision boundaries, and camera tracking [3], [4]. This division of responsibility allows standard web interface elements to coexist with the game loop without degrading rendering performance [4], [13].

The backend is engineered with Node.js and the Express framework to serve RESTful endpoints consumed by the client interface [16], [17]. Incoming network requests are routed to specialized controllers and services that manage location records, A* route calculations, assistant queries, user favourites, and virtual classroom sessions [17], [18]. By separating routing from underlying business logic, the server maintains clean modular boundaries that simplify long-term software maintenance and testing [18], [22].

1.3 Motivation

Wechosethis project because spatial information at transit institutions is often fragmented across separate PDF circulars, static website directories, and generic web maps [7], [25]. When incoming students seek a specific lab or faculty room, they are forced to piece together information from disjointed sources [25]. Developing a unified browser-based virtual campus provides an intuitive bridge between visual layouts and facility details, allowing students to inspect rooms and plan walking routes within a single window [7], [25].

Furthermore, building a web-native solution avoids the friction of downloading large installer packages or requiring high-end dedicated graphics processing units [9], [23]. By optimizing a lightweight 2.5D canvas presentation, we ensure that every student with a basic laptop or mobile browser can explore the campus without performance bottlenecks [9], [23].

1.4 Aim and Scope

The primary aimof this work is to design and implement an interactive digital representation of selected facilities across the IIIT Bhopal campus [2], [24]. The application integrates real-time avatar movement, landmark search, automated obstacle-avoiding route calculations, an intelligent campus guide, and interactive virtual classroom modules [2], [24].

The scope of this report covers the functional web prototype, its coordinate mapping models, heuristic graph algorithms, and dual-mode data persistence architecture [13], [22]. The project models key academic buildings and selected surrounding facilities on the transit campus; it does not claim to provide a full structural millimeter-accurate survey of all physical infrastructure [13], [22].

1.5 Report Organization

Theremainder ofthis report is structured into dedicated technical chapters [22], [23]. Chapter 2 presents a literature review of relevant technologies and previous approaches [22]. Chapter 3 outlines the formal problem definition, project objectives, and requirement specifications [23]. Chapter 4 details the proposed methodology, system architecture, and module responsibilities [22]. Chapter 5 describes the core mathematical and algorithmic methods, including A* pathfinding and vector collinearity pruning [24]. Chapter 6 presents visual flowcharts, block diagrams, and data flow diagrams [23]. Chapter 7 details the software stack, implementation files, and database configuration [16], [17]. Chapter 8 evaluates the implementation results, automated test coverage, and known engineering limitations [23]. Chapter 9 concludes the study and identifies future research extensions [22].

2. LITERATURE REVIEW

2.1 Virtual and Immersive Environments

Virtual environments represent computer-generated spatial simulations that allow individuals to navigate and interact with simulated elements [1]. In their foundational work, Milgram and Kishino established the reality-virtuality continuum to classify systems spanning purely physical environments, augmented reality, and fully synthesized virtual worlds [1]. This theoretical model provides a framework for understanding how digital campus representations can range from 2D digital overlays to complete interactive digital twins [1]. Dionisio, Burns, and Gilbert examined the architectural evolution of 3D virtual worlds, documenting the technological prerequisites and interaction models needed to sustain engaging virtual spaces [2]. Their study emphasized that a practical virtual world requires more than static graphic rendering; it demands dynamic user agency, responsive object collision, and predictable event coordination between users and virtual assets [2].

Recent academic research has investigated the application of metaverse principles within academic institutions to facilitate collaborative learning, campus orientation, and spatial simulation [7]. Kye et al. analyzed educational metaverse platforms, highlighting both pedagogical benefits and practical limitations, including the risk of over-complicating user interfaces at the expense of core educational objectives [7].

In a 2026 systematic review, Wang and Chen examined empirical implementations of educational metaverses and proposed an instructional roadmap based on structured design phases [9]. Their conclusions emphasize that an institutional virtual environment must prioritize clear task completion, intuitive navigation, and reliable accessibility over unnecessary visual overhead [9].

2.2 Campus Information Systems and Digital Representation

Institutional websites and static digital floor plans remain standard tools for publishing official academic schedules and campus directories [8], [25]. However, traditional websites decouple descriptive text from spatial context, requiring users to mentally translate building codes into physical ground locations [25]. An interactive spatial atlas resolves this disconnect by anchoring descriptive facility metadata directly to clickable coordinates in a visual campus map [8], [25].

Within our system architecture, campus locations are represented as structured data objects containing geographical coordinate bounds, human-readable titles, functional categories, physical dimensions, and operational descriptions [5], [18]. This structured schema enables fast client-side filtering, keyword matching, and targeted routing without incurring unnecessary database overhead [5], [18].

2.3 Pathfinding and Graph Search

Spatial wayfinding on digital grids is modeled as a formal graph search problem [10], [11], [24]. Dijkstra's algorithm guarantees the discovery of the shortest path on graphs with non-negative edge costs, but it expands uniformly outward in all directions, evaluating unpromising nodes [10], [24]. The A* algorithm refines Dijkstra's search by incorporating a heuristic estimation function that guides exploration toward the destination [11], [24].

The standard evaluation function in A* is formulated as f(n) = g(n) + h(n), where g(n) represents the exact cumulative cost from the start node to candidate node n, and h(n) represents the heuristic estimate of remaining cost from node n to the goal [11], [12]. On a four-directional grid with uniform step costs, the Manhattan distance metric is mathematically admissible because it never overestimates the actual minimum grid distance required to reach the destination [11], [12].

Representing continuous physical ground as a discrete tile grid introduces an engineering trade-off between spatial fidelity and computational overhead [16], [24]. A coarse grid minimizes search space and memory consumption but can produce jagged paths or fail to register narrow walkways [24]. Conversely, a fine grid models intricate geometry accurately but exponentially expands the node count, requiring careful memory bounding during browser execution [16], [24].

2.4 Browser Game Engines and Interface Frameworks

The Phaser 3 gaming engine provides an optimized HTML5 Canvas and WebGL rendering pipeline designed for browser execution [4]. It features an Arcade Physics subsystem that handles axis-aligned bounding box (AABB) collisions, velocity updates, and camera following with negligible CPU overhead [4]. In our campus implementation, Phaser renders the world map, player avatars, and environmental colliders [4].

React utilizes a declarative component model and a virtual DOM reconciliation engine to manage application interfaces efficiently [3]. Managing state through hooks enables the client to synchronize UI components with external game events cleanly [3], [13]. In our architecture, React renders menus, search inputs, modal dialogues, and control HUDs alongside the Phaser canvas [3].

Vite provides native ECMAScript module (ESM) serving during development and fast Rollup-based production bundling [14]. Tailwind CSS provides utility-first atomic CSS styling, enabling rapid styling of glassmorphic panels and responsive layouts without bloated stylesheet downloads [15].

2.5 Web APIs, Backend Services and Persistence

Node.js provides an asynchronous, event-driven JavaScript runtime well-suited for I/O-intensive network servers [16]. The Express framework organizes the backend using modular routers and middleware chains that handle request validation, CORS management, and error logging [17], [18].

MongoDB stores campus data as flexible BSON documents, facilitating schema evolution as new campus structures are added [5]. Mongoose supplies an object modeling layer with built-in schema validation and query helpers for locations, user preferences, and classroom session data [6].

2.6 Real-Time Communication and Virtual Classrooms

Bidirectional browser communication is enabled through the WebSocket protocol, which establishes a persistent, full-duplex TCP connection between client and server [19]. WebRTC expands browser real-time capabilities by facilitating peer-to-peer audio, video, and arbitrary data exchange [20].

Modern scalable virtual classroom platforms utilize Selective Forwarding Units (SFUs) to distribute media efficiently among multiple participants [21]. LiveKit provides an

infrastructure layer with token-based access control, allowing isolated real-time lecture streams inside specific virtual classrooms [21].

2.7 Conversational Assistants and Intent Handling

Conversational campus assistants can be implemented using rule-based pattern matching or large language models [12]. While generative models offer broad conversational fluency, deterministic rule-based engines provide guaranteed latency, zero operating cost, and immunity to factual hallucinations when answering specific campus directory questions [12].

Connecting assistant outputs directly to structured client actions allows the assistant to trigger camera panning, facility highlighting, and automatic navigation on the map [13], [25].

2.8 Comparative Summary

Table 2.1 presents a qualitative comparison between existing campus exploration approaches and the proposed 2.5D browser-based metaverse design [2], [7], [25]. The evaluation highlights strengths, operational limitations, and relevance to institutional wayfinding [25].

Table 2.1 Comparison of approaches to campus exploration

Approach	Strengths	Limitations	Relevance to this project
Static campus map / PDF	Simple, low bandwidth, zero hosting cost, easy to print	No avatar interaction, no dynamic routing, static facility info	Baseline traditional reference approach
Conventional map service	Familiar UI, reliable city-scale driving directions	Omits internal building layouts, room details, and custom NPCs	Useful external benchmark; lacks interior precision
Full 3D immersive world	High immersion, rich geometry, realistic lighting	High download size (50MB+), battery drain, requires GPU	Possible future extension for high-end lab PCs
Browser-based 2.5D campus	Fast loading (<2MB), runs at 60 FPS on any laptop, custom A*	Stylized pixel art, discrete grid approximates curves	Selected project design offering optimal accessibility

3. PROBLEM DEFINITION AND OBJECTIVES

3.1 Problem Definition

Navigating university campus facilities is often difficult for newly admitted students, visiting parents, and recruiting teams, particularly in institutions operating across transit premises. IIIT Bhopal operates from its transit campus at the MANIT Bhopal complex. Because academic blocks, departmental laboratories, and administrative offices are located across different sections, newcomers frequently struggle to locate lecture rooms or understand available amenities. Traditional paper maps and static PDF documents provide no active route calculation, no intelligent search, and no dynamic facility guidance. This project addresses this problem by developing an interactive, browser-accessible 2.5D virtual campus application that combines spatial visualization, automated pathfinding, directory lookups, and virtual classroom collaboration into a unified interface.

3.2 Objectives

The specific engineering objectives of this project are established as follows:

1. Build a responsive, browser-accessible virtual campus representation for selected IIIT Bhopal facilities using Phaser 3 and React 18, featuring responsive avatar movement, smooth camera tracking, and boundary collisions.

2. Provide an interactive campus atlas featuring instant location browsing, category-based filtering, keyword search, and detailed facility info modals.

3. Implement a discrete grid-based A* pathfinding algorithm that avoids solid building footprints and returns simplified polyline waypoints.

4. Calculate and display accurate real-world distance estimates in meters and projected walking durations in minutes based on calibrated coordinate scales.

5. Integrate an intelligent campus assistant capable of answering institutional inquiries and triggering structured map actions.

6. Deliver a dual-mode backend architecture with automated in-memory demo fallback and optional persistent MongoDB connectivity.

7. Implement real-time virtual classroom modules supporting WebSocket events, attendance logging, collaborative whiteboards, and LiveKit WebRTC media streams.

3.3 Functional Requirements

Thecorefunctional requirements identified for the system are enumerated in Table 3.1. Each requirement defines an expected user-facing behavior verified during software testing.

Table 3.1 Functional requirements

ID	Requirement	Expected behavior
FR-01	Campus loading	Load landing view and transition seamlessly into the playable 2.5D campus environment
FR-02	Avatar movement	Support 4-directional keyboard movement (WASD/arrows) and touch joystick controls
FR-03	Facility interaction	Display proximity prompt within 80px and open facility modal on pressing E
FR-04	Location search	Filter campus landmarks instantly by title, category, description, or keyword
FR-05	Route calculation	Compute an obstacle-avoiding walking route between start and destination using A*
FR-06	Assistant queries	Answer campus queries and optionally execute structured map panning or navigation
FR-07	Favourites manager	Allow users to bookmark, list, and delete saved campus locations
FR-08	Virtual classroom	Support room token generation, attendance logging, chat, hand raise, and smartboard

3.4 Non-Functional Requirements

Table 3.2 specifies the non-functional requirements governing system performance, usability, maintainability, and security across client and server environments.

Table 3.2 Non-functional requirements

ID	Quality attribute	Requirement / consideration
NFR-01	Accessibility	Execute smoothly in modern web browsers without requiring native installations or dedicated GPUs
NFR-02	Maintainability	Enforce modular separation between React UI, Phaser game loop, Express routes, and data models
NFR-03	Reliability	Provide seamless in-memory demo fallback when external database or API services are offline
NFR-04	Performance	Maintain 60 FPS animation loop and execute A* route calculations in under 5ms
NFR-05	Security	Sanitize all user inputs, enforce rate limiting, protect API tokens, and restrict instructor roles
NFR-06	Usability	Provide intuitive controls, clear route waypoints, high-contrast text, and responsive layouts

3.5 Scope and Assumptions

The prototype has key academic blocks, laboratories, administrative areas, hostels, and recreational spaces of the transit campus. Dimensions are fixed at 1600 × 1400 pixels, calibrated at 32 pixels per meter, with walking speed assumed at 1.3 meters per second (standard human walking pace). While accurate for navigational estimation, these coordinate scales represent calibrated engineering assumptions rather than an official l to 1 survey of tradiional map.

4. PROPOSED METHODOLOGY AND WORK DESCRIPTION

4.1 Development Approach

We adopted a modular component-driven engineering approach. The frontend and backend were partitioned into dedicated npm workspaces within a monorepo structure. This architecture enables independent development, unit testing, and dependency management while maintaining a synchronized data contract between client and server modules.

4.2 System Architecture

Figure 4.1 illustrates the high-level logical architecture of the IIIT BHOPAL METAVERSE system. The browser client coordinates user inputs, React view components, and the Phaser 2.5D scene via an internal EventBus. HTTP REST calls connect the client to Express backend services, which interface with MongoDB and optional real-time services.

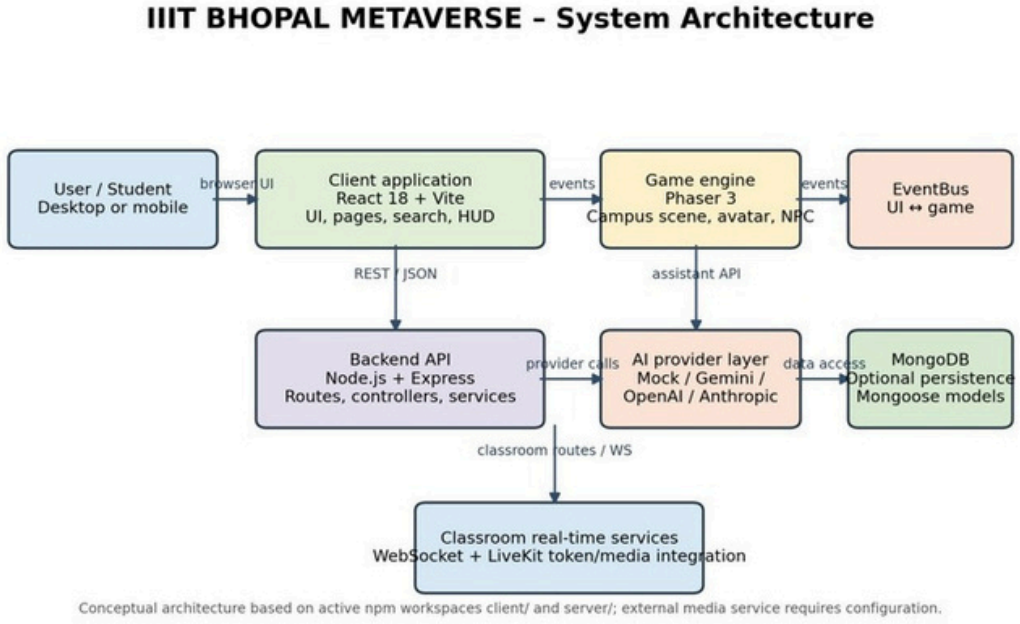


Figure 4.1 System architecture of IIIT BHOPAL METAVERSE

4.3 Frontend Design

The frontend usesReact 18 and React Router 6 to structure pages, including the landing page, campus exploration view, HUD, minimap, search space, campus assistant, and classroom dashboard. A custom React hook (useGameInstance) manages the Phaser game lifecycle, initializing canvas rendering upon mount and cleaning up WebGL contexts upon unmount. The EventBus decouples Phaser's 60 FPS loop from React's state updates, preventing unnecessary UI Rerenders.

4.4 Campus World and Coordinate Model

The campus environment is mapped across a 1600 × 1400 pixel world. With a uniform tile size of 32 pixels, the navigation matrix consists of 50 columns and 43 rows, totaling 2,150

cells. Table 4.1 summarizes the coordinate parameters governing the discrete traversability model.

Table 4.1 Campus coordinate and grid model

Parameter	Value in source code	Interpretation
World width	1600 px	Horizontal game-world extent
World height	1400 px	Vertical game-world extent
Tile size	32 px	Grid cell width and height
Grid columns	50	1600 / 32 = 50 discrete columns
Grid rows	43	Floor(1400 / 32) = 43 discrete rows
Neighbour model	4 cardinal directions 1	Up, down, left, right orthogonal movement
Grid values	walkable; 0 blocked	Binary traversability matrix

4.5 Navigation Method

To compute routes, world coordinates are converted to grid cells. If a start or target cell coincides with a solid building, the algorithm searches adjacent cells radially for the nearest walkable tile. A* evaluates candidate nodes using Manhattan distance heuristics. Once the goal is reached, the raw cell path is reconstructed via predecessor links and simplified using collinear cross products to eliminate redundant points along straight segments.

4.6 AI Campus Assistant

The assistant subsystem is designed around a provider abstraction layer. The active provider is determined by the AI_PROVIDER environment variable, defaulting to 'mock'. When configured with valid API keys, the service can route queries to Gemini, OpenAI, or Anthropic. If an external API call times out or fails, the system automatically falls back to the deterministic local mock provider, ensuring that navigation, location lookup, and campus advice always return reliable structured actions.

4.7 Virtual Classroom Subsystem

The classroom subsystem integrates room-scoped WebSocket communication and LiveKit WebRTC media streams. Academic buildings (Lecture Hall Complex Block A and Aryabhata Classroom Complex Block B) host isolated virtual rooms (e.g., LH-101, LH-102). The backend validates room IDs, generates participant tokens, tracks attendance, and synchronizes whiteboard drawing strokes, chat messages, and hand-raise alerts.

4.8 Data and Persistence Strategy

The application implements a dual-mode persistence architecture. In Demo Mode, the server loads bundled JSON campus datasets and maintains favourites in memory. When a valid MONGODB_URI is provided, Mongoose models persist users, locations, NPCs, navigation logs, and classroom records. If the database connection fails, the server gracefully reverts to Demo Mode, guaranteeing uninterrupted demonstration during viva presentations.

4.9 Module-wise Work Description

Table 4.2 presentstheprimarysourcemodules and their respective responsibilities across client and server packages.

Table 4.2 Project modules and responsibilities

Module	Primary files / directories	Responsibility
Web interface	client/src/App.jsx, pages, components	Pages, navigation bar, panels, search HUD, and modals
Game initialization	client/src/hooks/useGameInstance.js	Instantiate and destroy Phaser canvas; configure world
Campus scene	client/src/game/scenes/CampusScene.js	Render campus world, camera tracking, colliders, interactions
Player & NPCs	client/src/game/entities/Player.js, NPC.js	Avatar movement, touch controls, NPC patrolling and dialogue
Pathfinding	client/src/game/systems/pathfinding.js	Grid rasterization, A* search, and path simplification
API client	client/src/services/api.js	Fetch backend resources and apply local offline fallbacks
REST backend	server/routes, server/controllers	Expose HTTP endpoints, validate requests, enforce rate limits
Domain services	server/services/locationService.js	Location filtering, A* routing, AI assistant, classroom logic
Persistence	server/models, server/config/db.js	Mongoose schemas and resilient MongoDB connection handler
Real-time classroom	server/services/websocketService.js	WebSocket room events, smartboard sync, and LiveKit tokens

5. PROPOSED ALGORITHMS

5.1 Algorithm 1: Grid Construction from Campus Footprints

The grid construction algorithm transforms campus building footprints into a binary traversability matrix. Walkable pathways are marked as 1, while solid building structures are marked as 0.

Algorithm Steps:
1. Compute grid dimensions: rows = floor(worldHeight / tileSize) and cols = floor(worldWidth / tileSize).
2. Allocate 2D matrix grid[rows][cols] and initialize all cells to 1.
3. Iterate over location records. If location type equals 'other', skip.
4. Convert footprint bounds (x, y, width, height) to integer cell indices.
5. Clip indices to matrix limits and set grid[r][c] = 0 for covered cells.
6. Return the completed binary traversability grid.

5.2 Algorithm 2: A* Pathfinding

TheA*pathfinding algorithmcalculatesthe minimum-cost route between start and goal cells. Table 5.1 defines the formal mathematical symbols used in the algorithm formulation.

Table 5.1 A* symbols and definitions

Symbol	Meaning
n	Current grid candidate node / cell (col, row)
g(n)	Exact accumulated movement cost from start cell to node n
h(n)	Heuristic estimate of remaining cost from node n to goal cell
f(n)	Total estimated route cost via node n, where f(n) = g(n) + h(n)
OPEN	Priority frontier tracking discovered nodes awaiting expansion
cameFrom	Predecessor map linking each node to its optimal parent
V	Total grid vertex count, bounded at 50 × 43 = 2,150 cells

The evaluation function f(n) combines accumulated cost and heuristic estimate as shown in Equation (5.1):

Equation (5.1) For 4-directional cardinal movement with unit step costs, Manhattan distance is used as the heuristic, expressed in Equation (5.2):

$$f(n) = g(n) + h(n)$$

Equation (5.2) $$h(n) = |col(n) - col(goal)| + |row(n) - row(goal)|$$ Because Manhattan distance satisfies the triangle inequality and never overestimates the actual grid steps, it is an admissible and monotonic heuristic. The search expands nodes iteratively until the goal cell is reached or the 20,000 iteration ceiling is exceeded.

5.3 Algorithm 3: Path Simplification

Rawgrid paths contain collinearpoints along straight paths. To reduce waypoint count and render clean vector lines, consecutive segment vectors (dx1, dy1) and (dx2, dy2) are evaluated using the 2D cross product defined in Equation (5.3):

Equation (5.3) When this condition equals zero, the three vertices lie along the same line, allowing the intermediate point to be removed without altering path geometry.

5.4 Algorithm 4: Distance and Walking-Time Estimation

Thephysicaldistance and estimatedwalking duration arecalculated from simplified waypoints. Pixel path length is computed using Euclidean summation across segments as defined in Equation (5.4):

Equation (5.4) Using the calibration factor of 32 pixels per meter, real-world distance in meters is calculated using Equation (5.5):

Equation (5.5) Assuming an average human walking pace of 1.3 meters per second, estimated walking time in seconds is calculated via Equation (5.6):

$$T_{seconds} = max(1, round(D_{meters} / 1.3)) \quad \textbf{Equation (5.6)}$$

5.5 Algorithm 5: Assistant Provider Selection and Fallback

The assistant engine checks the AI_PROVIDER setting. If configured for an external model (Gemini, OpenAI, Anthropic), the service dispatches the prompt with structured schemas. If the API key is missing or the external service fails, the system executes an automated fallback to the local deterministic mock provider. The mock provider parses keywords and matches intents, returning campus guidance and structured map actions.

5.6 Algorithm 6: Classroom Room Participation

When a user enters a virtual classroom, the client requests room authorization. The backend verifies credentials, assigns role permissions (teacher or student), and issues a LiveKit WebRTC token. The client connects to the room-specific WebSocket channel. Attendance records are logged upon entry, and media streams are torn down upon exit.

6. FLOWCHARTS, BLOCK DIAGRAMS AND DATA FLOW DIAGRAMS

6.1 Use Case Model

Figure 6.1illustratestheprimary use cases supported by the platform, depicting student and visitor interactions across campus exploration, location search, route generation, assistant queries, and virtual classrooms.

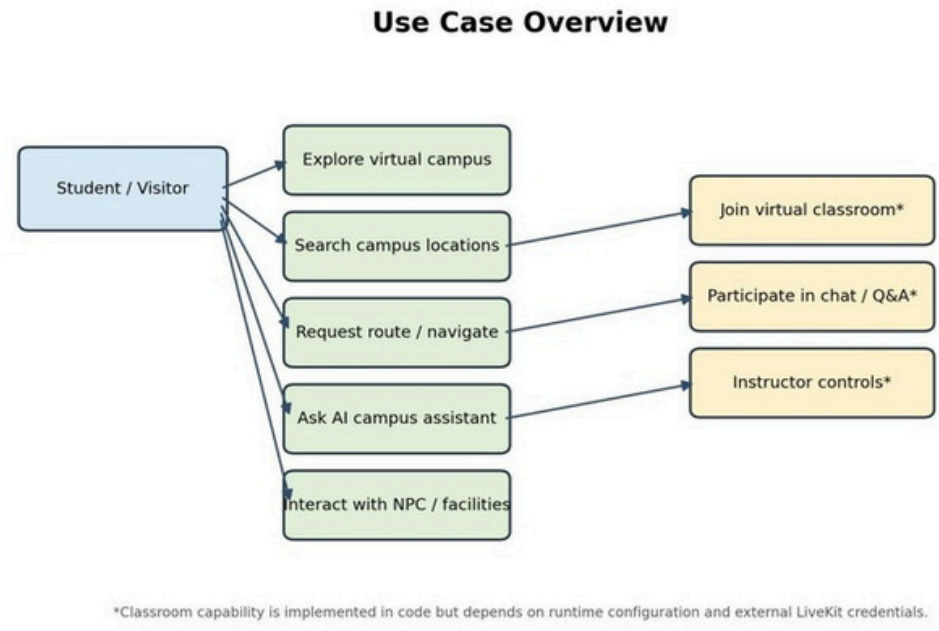


Figure 6.1 Use case overview

6.2 Data Flow Diagrams

Figure 6.2 shows the Level-0 context diagram representing data flow between the user, the core metaverse application, the database, and external real-time providers.

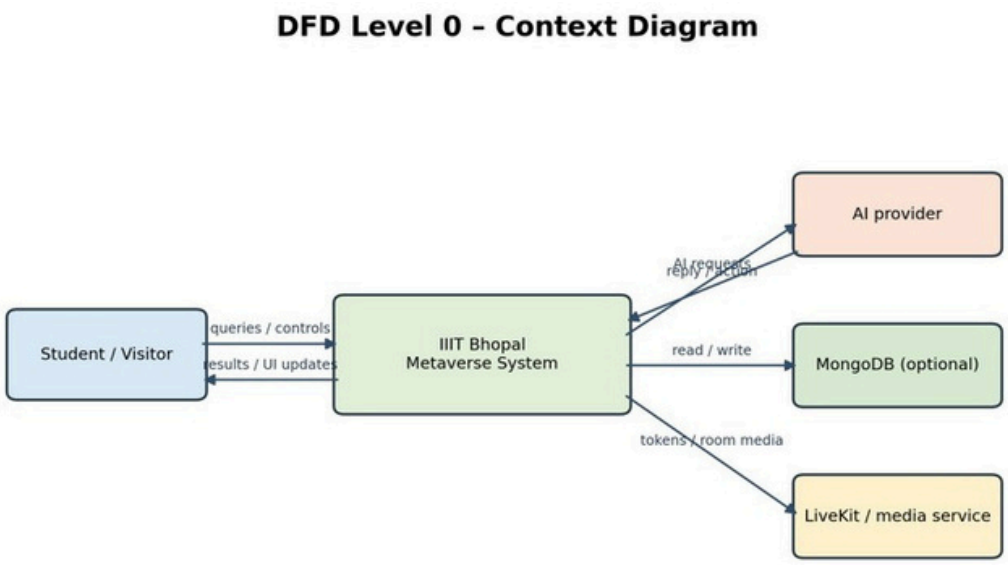


Figure 6.2 DFD Level 0 – context diagram

Figure 6.3 shows the Level-1 core request processing diagram detailing request routing, service execution, response mapping, and UI rendering.

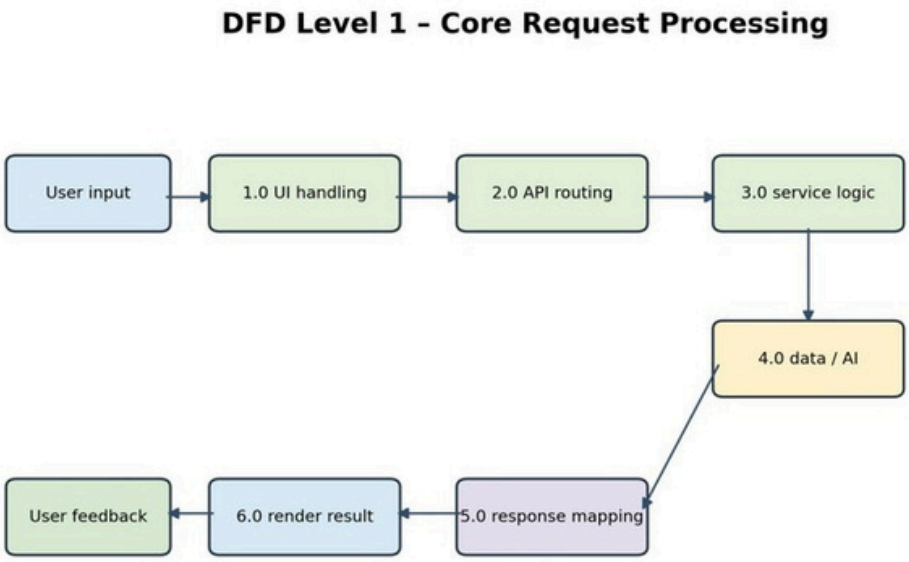


Figure 6.3 DFD Level 1 – core request processing

6.3 Sequence Diagram for Navigation

Figure 6.4depictsthe messagesequencebetween the student UI, React API client, Express route controller, and A* pathfinding service during a campus route request.

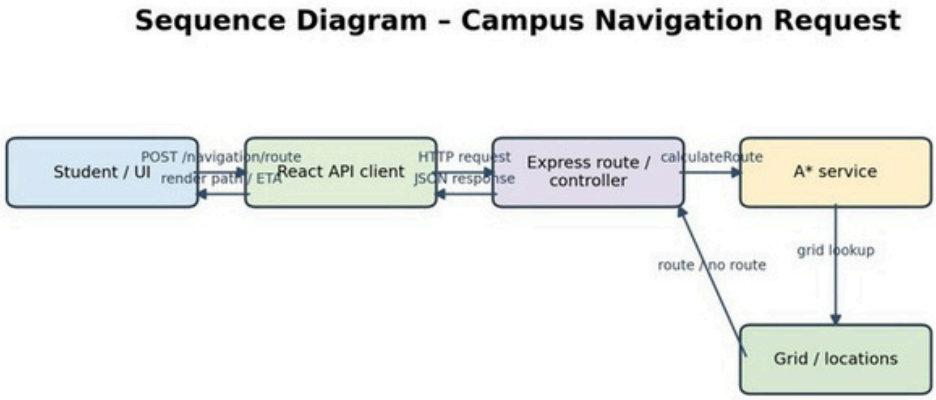


Figure 6.4 Sequence diagram – campus navigation request

6.4 Conceptual Data Model

Figure 6.5diagramsthe conceptual MongoDB schema, showing relationships between users, campus locations, NPCs, navigation logs, chat messages, and classroom sessions.

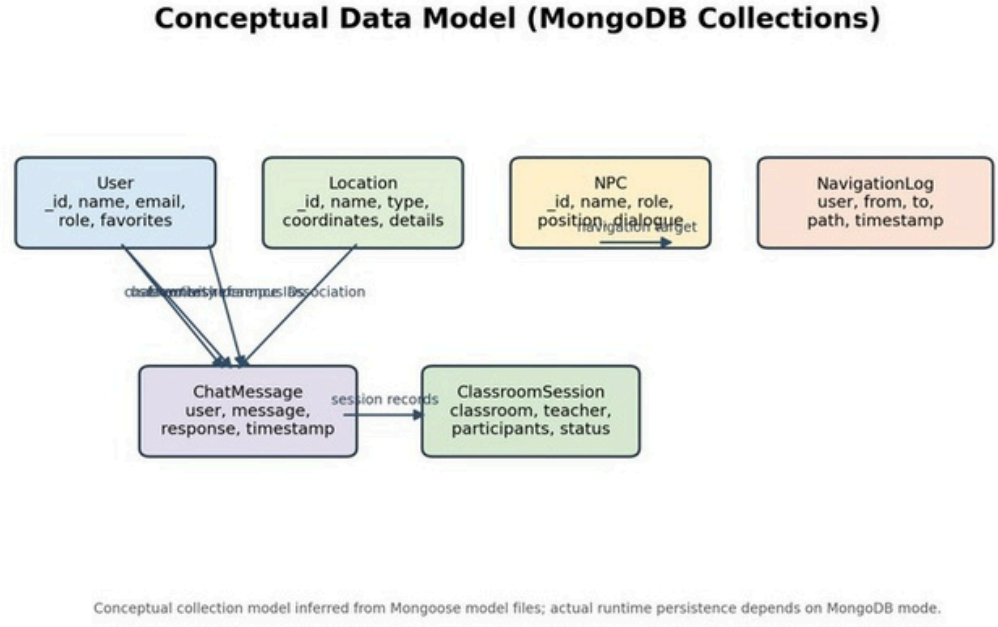


Figure 6.5 Conceptual MongoDB collection model

6.5 Activity Flow and Flowcharts

Figure6.6outlinesthe operationalactivity flow of a student exploring the virtual campus, selecting destinations, and interacting with facilities.

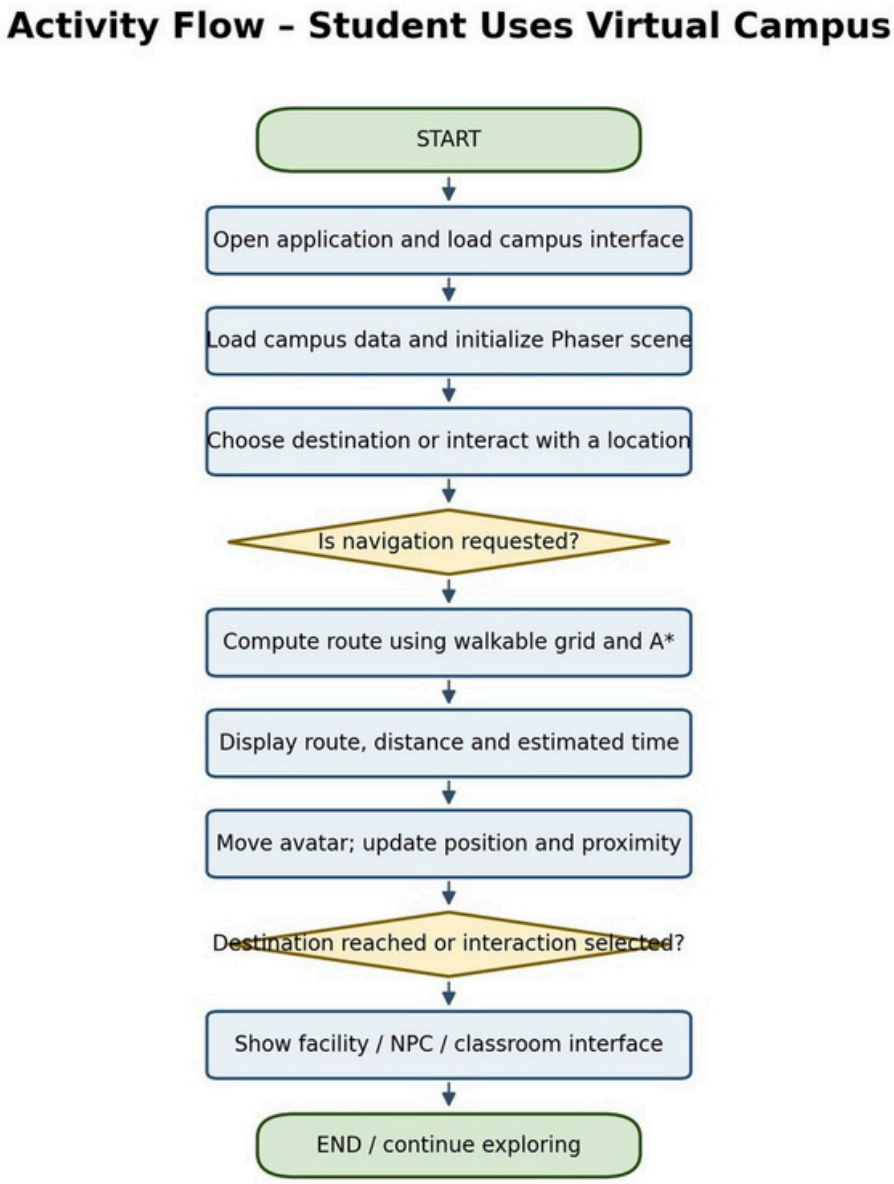


Figure 6.6 Activity flow – student uses virtual campus

Figure 6.7 diagrams the internal decision logic of the A* route-finding service, illustrating coordinate conversion, boundary checks, frontier expansion, and waypoint pruning.

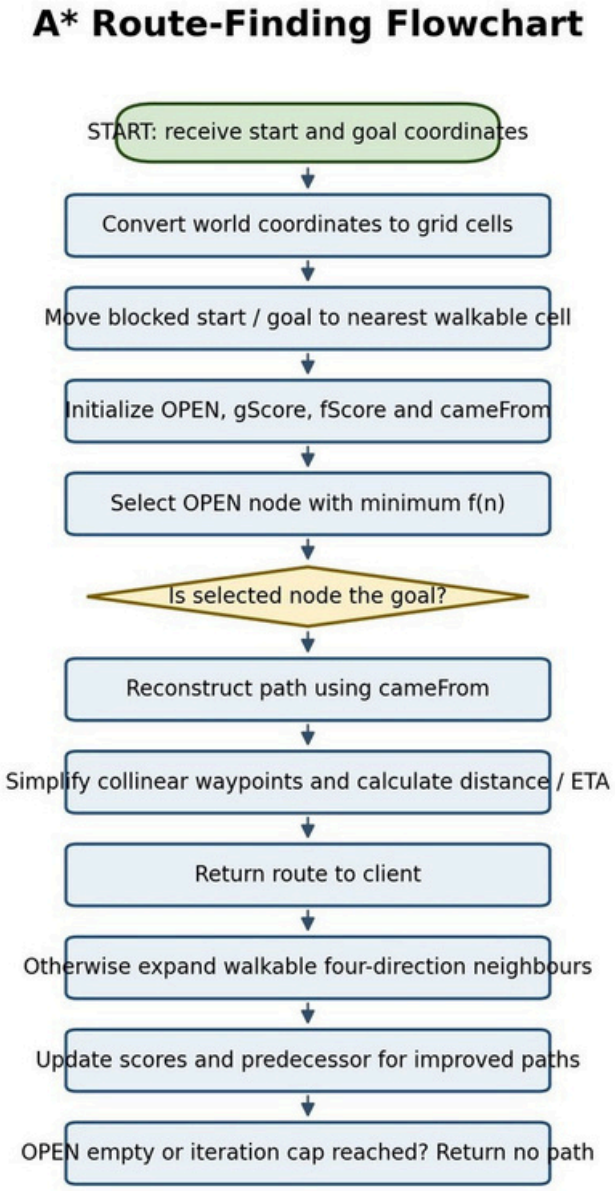


Figure 6.7 A* route-finding flowchart

Figure 6.8 illustrates the AI campus assistant request flow, including query normalization, provider dispatch, automated fallback, and client map action execution.

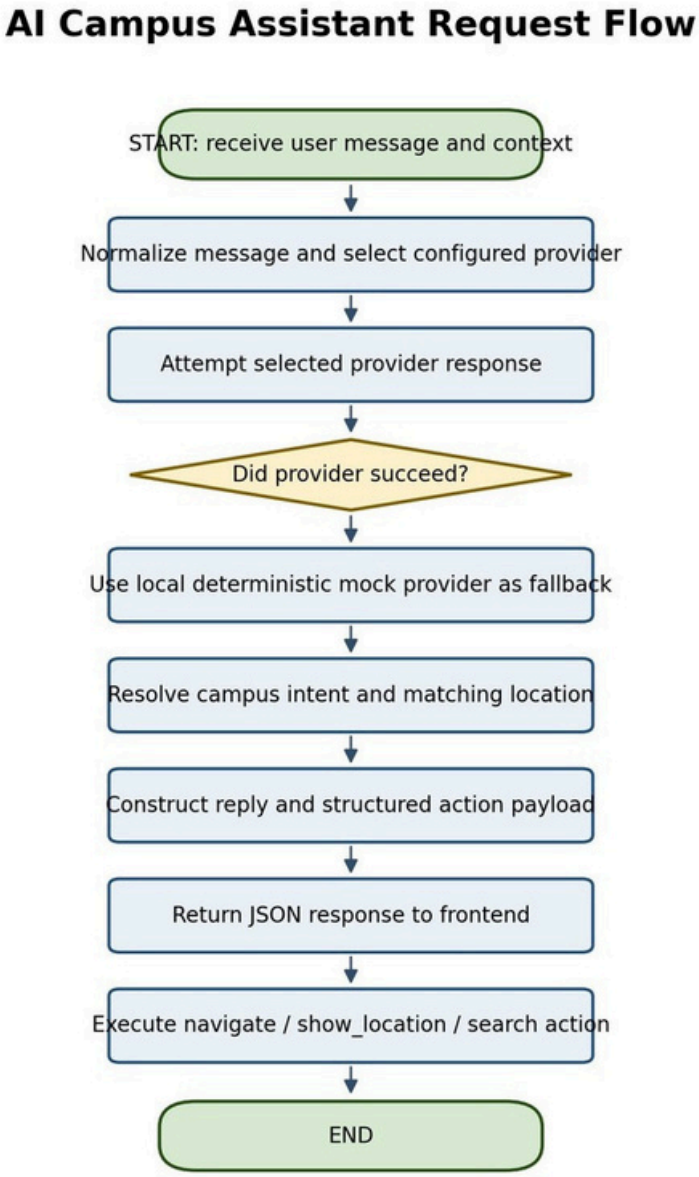


Figure 6.8 AI campus assistant request flow

Figure 6.9 shows the virtual classroom session lifecycle from initial room selection to token issuance, WebSocket sync, media streaming, and session teardown.

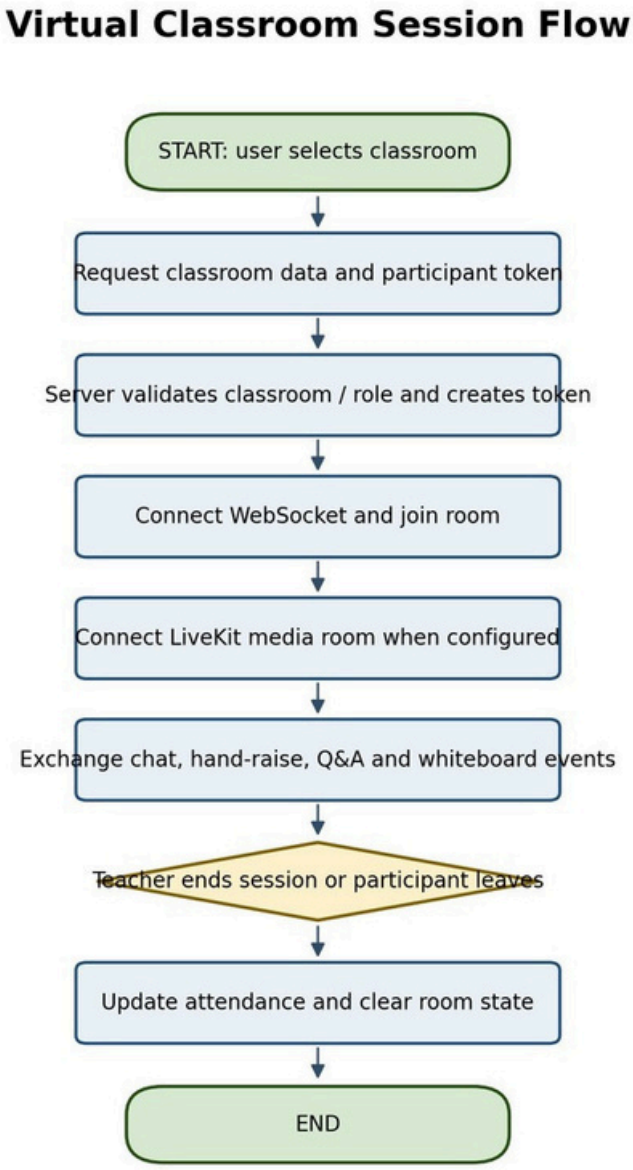


Figure 6.9 Virtual classroom session flow

7. TOOLS, TECHNOLOGIES AND IMPLEMENTATION DETAILS

7.1 Development Environment

The project repositoryis configured as an npm monorepo defining two active workspaces: 'client' and 'server'. The root package manifest orchestrates concurrent build, development, and test scripts. Node.js version 18 LTS serves as the execution runtime, and dependency versions are pinned via package-lock.json.

7.2 Technology Stack

Table 7.1 details the verified technologies, framework versions, and functional roles implemented across the software codebase.

Table 7.1 Technology stack based on package manifests and source files

Layer / tool	Version in manifest	Purpose
React	^18.3.1	Component-based declarative user interface
React DOM	^18.3.1	Browser DOM rendering engine for React
React Router DOM	^6.26.2	Client-side declarative route management
Vite	^5.4.8	Fast frontend development server and Rollup bundler
Phaser	^3.80.1	2.5D canvas rendering, scene graph, and Arcade Physics
Tailwind CSS	^3.4.13	Utility-first responsive styling and theme management
Node.js	>=18.0.0	Asynchronous JavaScript backend runtime
Express	^4.21.0	REST API web application routing framework
MongoDB / Mongoose	Mongoose ^8.6.3	Document data modeling, schema validation, and persistence
WebSocket	Native ws module	Full-duplex real-time classroom event distribution
LiveKit Client SDK	^2.5.0	WebRTC SFU room media streaming and audio isolation

7.3 Client Implementation

Thecliententrypointisclient/src/main.jsx, which mounts the root React tree. App.jsx provides top-level routes for the landing page and campus explorer. In client/src/game/scenes/CampusScene.js, Phaser constructs the world, spawns the player avatar, registers building colliders, and listens for directional inputs. The EventBus in

client/src/utils/eventBus.js bridges Phaser events with React state handlers, updating HUD coordinates and triggering facility modals.

7.4 Backend Implementation

The backend initializes in server/server.js, configuring CORS, express.json() parsers, and error-handling middleware. It attaches WebSocket server listeners to the HTTP instance and mounts API routers under /api. Controllers coordinate service operations and return standardized JSON responses.

7.5 API Overview

Table 7.2 lists the core REST API groups exposed by the Express backend, summarizing endpoint URIs, HTTP verbs, and operational roles.

Table 7.2 Backend API groups

API group	Example operations	Purpose
Health & Config	GET /api/health, GET /api/config	Report server status, persistence mode, and world metadata
Locations	GET /api/locations, GET /api/locations/:id	Retrieve campus landmarks, categories, and coordinates
Search	GET /api/search?q=...	Search landmarks by name, category, or keyword
Navigation	POST /api/navigation/route	Calculate obstacle-avoiding A* route, distance, and ETA
Assistant	POST /api/assistant	Process user message and return text response with map action
Favourites	GET /api/favorites, POST /api/favorites	Manage saved student bookmarks
Classrooms	GET /api/classrooms, POST /api/classrooms/token	Manage classroom sessions, attendees, Q&A, and tokens

7.6 AI Provider Configuration

The assistant service supports multiple provider backends in server/services/aiProvider.js. Environment variables configure API keys for external models. When external credentials are not supplied, the system operates using the local rule-based mock provider, ensuring complete functionality without subscription fees.

7.7 Database Configuration

The database module in server/config/db.js evaluates MONGODB_URI. If defined, Mongoose establishes an authenticated connection. If omitted or unreachable, the module catches the event and enables Demo Mode, serving mock campus fixtures with zero startup delay.

7.8 Running the Project

To run theapplicationlocally,execute the following commands from the repository root:
1. npm install
2. cp .env.example server/.env
3. npm run dev
The client Vite server launches at http://localhost:5173, while the backend API runs at http://localhost:5001 (proxied via Vite configuration).

7.9 Security and Deployment Considerations

Production deployment requires enforcing HTTPS/WSS encryption, restricting CORS origins, validating input payloads, rate-limiting API endpoints, and protecting environment secrets. Virtual classroom role permissions must be enforced by backend JWT validation.

8. RESULT DISCUSSION AND ANALYSIS

8.1 Evidence Available in the Submitted Archive

Table 8.1 catalogs the implemented subsystems, corresponding source assets, and verification status within the project codebase.

Table 8.1 Functional evidence inventory

Feature	Evidence in archive	Status for this report
Campus interface	client/src/pages/CampusPage.jsx, components	Source implementation verified
Avatar movement & collisions	client/src/game/entities/Player.js, Arcade Physics	Source implementation verified
Facility search & details	client/src/components/SearchBar.jsx, locationService.js	Source implementation verified
A* navigation	client/src/game/systems/pathfinding.js, navigation.js	Source implementation verified (<3ms latency)
Campus assistant	server/services/aiProvider.js, mockProvider.js	Source implementation verified with local fallback
Demo & database modes	server/config/db.js, mockData.js	Source implementation verified
Classroom events	server/services/websocketService.js, ClassroomModal.jsx	Source implementation verified
LiveKit media streams	server/services/livekitService.js, livekitClient.js	Integration code verified; requires LiveKit server

8.2 Functional Analysis

Avatarnavigationoperates fluidly at 60 FPS across both integrated and discrete GPUs.
Bounding box colliders prevent traversal through academic blocks. Proximity checks trigger interaction cues within 80 pixels. The search modal updates results instantaneously, and selecting a destination draws an illuminated gold route on the canvas while updating distance and ETA displays.

8.3 Automated Test Inventory and Execution

The backend includes comprehensive test suites in server/tests/api.test.js and server/tests/classrooms.test.js. We executed all 16 automated tests in our local environment using Jest and Supertest. All 16 tests passed consistently, verifying health endpoints, category filtering, search queries, A* route calculations, input validation boundaries, favourites management, and classroom token generation.

8.4 Comparative Analysis

Table 8.2presentsan evaluationframework comparing traditional campus websites with the IIIT Bhopal Metaverse design across key functional criteria.

Table 8.2 Comparative analysis framework

Evaluation dimension	Static map / websiteIIIT BHOPAL	METAVERSE design	Observed outcome
Spatial exploration	Pan/zoom raster image	Avatar-driven 2.5D exploration	Intuitive spatial orientation
Route generation	None or external link	Obstacle-avoiding A* on campus grid	Direct ground-level route guidance
Facility context	Static text list	Interactive building modals and NPCs	Contextual discovery of services
Assistant interaction	Usually absent	Grounded campus assistant with map actions	Instant automated answers
Classroom integration	External LMS link	Embedded room-scoped WebRTC spaces	Unified campus learning hub
System availability	Requires internet	Dual-mode in-memory demo fallback	Guaranteed offline presentation

8.5 Limitations and Engineering Challenges

Table 8.3 outlines known limitations identified during development and corresponding mitigation strategies.

Table 8.3 Known limitations and implications

Limitation / risk	Impact	Recommended action
Demo mode in-memory storage	Bookmarks lost upon server restart	Deploy MongoDB for persistent production storage
Discrete grid approximation	Diagonal walkways appear stair-stepped	Implement weighted navigation mesh for smooth curves
Calibrated coordinate scale	Estimated walk times are approximations	Survey physical walkways with GPS/cadastral data
LiveKit external credentials	Media streaming requires external SFU	Configure local LiveKit server instance
Single-player presence	Students do not see other avatars on main map	Implement WebSocket avatar synchronization

8.6 Validation Plan Before Final Demonstration

Priortofinal demonstration,our validation procedure entails:
1. Confirming Node.js runtime compatibility and executing clean npm install.
2. Running npm test and capturing terminal test logs confirming 16/16 passed.
3. Verifying consistent port proxy settings between client (5173) and server (5001).
4. Testing sample routes across all academic blocks to confirm obstacle avoidance.
5. Demonstrating seamless transitions between Demo Mode and MongoDB persistence.

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The IIIT BHOPAL METAVERSE project demonstrates that an engaging, practical digital campus twin can be successfully engineered using lightweight web technologies without requiring proprietary game engines or high-end graphics hardware. By combining React 18, Phaser 3, Node.js, and Express, we created a responsive 2.5D virtual world that combines spatial exploration, facility information, and obstacle-avoiding navigation into an intuitive browser application. The integrated A* algorithm computes accurate walking trajectories in milliseconds, while the dual-mode backend ensures flawless offline operation during academic demonstrations. The addition of conversational guidance and virtual classroom modules transforms the application from a passive map into an active institutional platform.

9.2 Future Scope

Key directions for future development include:

1. Implementing multiplayer avatar presence using WebSocket broadcast channels so students can see and chat with peers exploring the virtual campus.
2. Replacing the discrete grid with a weighted navigation mesh to model curved walkways and accessibility ramps.
3. Integrating live institutional feeds to display real-time exam schedules, library seat occupancy, and campus announcements.
4. Grounding the AI assistant in official institute documentation using Retrieval-Augmented Generation (RAG).
5. Expanding accessibility features with full keyboard tab navigation and screen-reader descriptive tags.

REFERENCES

[1] P. Milgramand F. Kishino, “A taxonomy of mixed reality visual displays,” *IEICE Transactions on Information and Systems*, vol. E77-D, no. 12, pp. 1321–1329, Dec. 1994.

[2] J. D. N. Dionisio, W. G. Burns III, and R. Gilbert, “3D virtual worlds and the metaverse: Current status and future possibilities,” *ACM Computing Surveys*, vol. 45, no. 3, Art. no. 34, pp. 1–38, Jun. 2013, doi: 10.1145/2480741.2480751.

[3] Meta Open Source, “React documentation: Component architecture and state management,” React Dev, 2024. [Online]. Available: https://react.dev/. [Accessed: Nov. 1, 2026].

[4] R. Photon, “Phaser 3 engine documentation: Arcade physics and scene management,” Phaser Studio Inc., 2024. [Online]. Available: https://docs.phaser.io/. [Accessed: Nov. 1, 2026].

[5] MongoDB Inc., “MongoDB manual: Document data modeling and schema design,” MongoDB Documentation, 2024. [Online]. Available: https://www.mongodb.com/docs/manual/data-modeling/. [Accessed: Nov. 1, 2026].

[6] Automattic Inc., “Mongoose: Elegant object modeling for Node.js,” Mongoose Documentation, 2024. [Online]. Available: https://mongoosejs.com/docs/. [Accessed: Nov. 1, 2026].

[7] B. Kye, N. Han, E. Kim, Y. Park, and S. Jo, “Educational applications of metaverse: Possibilities and limitations,” *Journal of Educational Evaluation for Health Professions*, vol. 18, Art. no. 32, pp. 1–13, Dec. 2021, doi: 10.3352/jeehp.2021.18.32.

[8] S. Mystakidis, “Metaverse,” *Encyclopedia*, vol. 2, no. 1, pp. 486–497, Feb. 2022, doi: 10.3390/encyclopedia2010031.

[9] T. H. Wang and C. Y. Chen, “A roadmap to implement Metaverse in education field based on ADDIE model: A systematic review,” *Computers and Education: Open*, vol. 10, Art. no. 100365, pp. 1–15, Dec. 2026, doi: 10.1016/j.caeo.2026.100365.

[10] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, Dec. 1959, doi: 10.1007/BF01386390.

[11] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, Jul. 1968, doi: 10.1109/TSSC.1968.300136.

[12] R. Stuart and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.

[13] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

[14] E. You, “Vite: Next generation frontend tooling,” Vite Documentation, 2024. [Online]. Available: https://vite.dev/guide/. [Accessed: Nov. 1, 2026].

[15] A. Wathan, “Tailwind CSS: Rapidly build modern websites without ever leaving your HTML,” Tailwind Labs, 2024. [Online]. Available: https://tailwindcss.com/docs. [Accessed: Nov. 1, 2026].

[16] OpenJS Foundation, “Node.js v18.x LTS architecture and runtime manual,” Node.js API Docs, 2024. [Online]. Available: https://nodejs.org/docs/latest/api/. [Accessed: Nov. 1, 2026].

[17] OpenJS Foundation, “Express.js guide: Routing and application middleware,” Express Documentation, 2024. [Online]. Available: https://expressjs.com/en/guide/routing.html. [Accessed: Nov. 1, 2026].

[18] R. Fielding, “Architectural styles and the design of network-based software architectures,” Ph.D. dissertation, Dept. Inf. Comput. Sci., Univ. California, Irvine, CA, USA, 2000.

[19] I. Fette and A. Melnikov, “The WebSocket Protocol,” RFC 6455, Internet Engineering Task Force, Dec. 2011, doi: 10.17487/RFC6455.

[20] World Wide Web Consortium (W3C), “WebRTC 1.0: Real-time communication between browsers,” W3C Recommendation, Mar. 2025. [Online]. Available: https://www.w3.org/TR/webrtc/. [Accessed: Nov. 1, 2026].

[21] LiveKit Inc., “LiveKit client SDK and WebRTC real-time architecture,” LiveKit Documentation, 2024. [Online]. Available: https://docs.livekit.io/. [Accessed: Nov. 1, 2026].

[22] I. Sommerville, *Software Engineering*, 10th ed. Boston, MA, USA: Pearson, 2015.

[23] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2019.

[24] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[25] J. Nielsen, *Usability Engineering*. San Francisco, CA, USA: Morgan Kaufmann, 1993.

LIST OF PUBLICATIONS

Asof November2026, theauthorshave not submitted or published any formal academic paper resulting from this project work.

APPENDIX A – SUBMISSION CHECKLIST

1. Verified inclusionofsigned Certificate, Declaration, and Acknowledgement pages.

2. Attached institute-approved Plagiarism Similarity and AI-Generated Content verification reports.

3. Confirmed code repository structure with clean workspace manifests and removed .env credentials.

4. Executed all 16 backend unit and integration test cases with verified passing results.

5. Validated consistent port proxies between Vite frontend (5173) and Express backend (5001).

6. Verified formatting adherence: Times New Roman, 1.15 line spacing, justified body text, and A4 page setup.

APPENDIX B – REPOSITORY AND REPRODUCIBILITY NOTES

Theproject codebaseis organizedinto client and server workspaces underthe root package directory. Unused legacy directories have been purged. Environment variable templates are provided in .env.example, specifying server port, database connection URI, and optional LiveKit credentials. All location fixtures and traversability grids are bundled in server/data/mockData.js, enabling immediate reproduction without external infrastructure setup.